⑥ Membership operators:-

=> The purpose of membership operators is that "To check the existence of a value in an iterable object"

- An iterable object is one which contains more than one value (sequence Type, List type, set Type, Dict Type).
- A non iterable object is one which contains only one value (All fundamental Types, None Type)

=> In python programming, we have two types of membership operators They are: ① in operator ② not in operator

① in operator:-

syntax :- value in iterable_object

=> Here "in" operators returns True provided the specified value present in iterable object.

=> Here "in" operators returns False provided the specified value Not present in iterable object.

② not in operator:-

syntax : value not in iterable_object

=> Here "not in" operators returns True provided the specified value Not present in iterable object.

=> Here "not in" operator returns false provided the specified value present in iterable object.

Examples

```
>>> s = "PYTHON"
>>> print (s, type(s)) ——— PYTHON <class str>
>>> "p" in s  - - - True
>>> "O" in s  - - - T
```

```
>>> "N" not in s  - - - - F
>>> "p" in s  - - - - F
>>> "p" not in s  - - - - T
>>> "t" in s  - - - - T
>>> "T" not in s  - - - - F.
```

```
>>> s = "PYTHON"
>>> print(s, type(s)) - - - - PYTHON <class 'str'>
>>> "PYT" in s  - - - T
>>> "PXT" not in s  - - T
>>> "pyt" in s  - - - F
>>> "PTO" in s  - - - F
>>> "poN" not in s  - - - T
>>> "pyT" not in s  - - - T
>>> "pYt" not in s  - - - T
>>> "NoH" in s  - - - F
>>> "NoH" not in s[::-1] - - - F.
```

```
>>> s = "PYTHON"
>>> print(s, type(s)) - - - PYTHON <class 'str'>
>>> "HoN"[::-1] in s[::-1] - - - - T
>>> "PTO" in s[::2]  - - - T
>>> "PTO" not in s[::2][::-1] - - - T
>>> "PTO"[::-1] in s[::2] - - - - F
```

```
>>> lst = [10, "Rossum", 23.45, 2+3j]
>>> print(lst) - - - - [10, Rossum, 23.45, 2+3j]
>>> "Ross" in lst - - - F
>>> "Ross" not in lst[1] - - - F
>>> "Ross" in lst[1] - - - T
>>> "Rossum"[::-1] not in lst[-3][::-1] - - - F
>>> "Rossum"[::-1] in lst[-3][::-1] - - - T.
>>> (2+3j) in lst[-1] - - - Type error.
>>> lst[-1] in lst - - - True.
```

```
>>> (2+3j) in str(list[-1]) ---- TypeError.
>>> 12 in "123"  — TypeError
>>> "12" in "123"  —— T.
```

```
>>> d = {10:"Apple", 20:"mango", 30:"kiwi"}.
>>> "Apple not in d  - - - T
>>> for val in d:
        print(val)
10
20
30.
>>> 10 in d —  — - T
>>> "20" not in d  - -- T
>>> "20" in d  — - - F
>>> "20" in d.keys()  - - T
>>> d[10][::-1] in d.get(10) — - -- F
>>> "1" not in "123"[1]  - - T
```

```
>>> "mis" in "MISSSISSIPPI" — - - T
>>> "IIp" not in "MISSISSIPPI"[::-1] — - T
>>> "IIp"[-1] not in "MISSISSIPPI" — - - T.
```

Day 32

7. Identity operators (python command prompt)

→ The purpose of identity operators is that "To compare the memory Address of two objects".

=> In python programming we have two types of identity operators. They are:

1. is
2. is not

1. is operator

=> syntax : object1 is object2

- Here "is" operator returns True provided the Both the objects contains same memory address.

=) Here "is" operator returns False provided the Both the objects contains Different memory Address.

② is not operator :-

syntax : objects1 is not object2

=) Here "is not" operator returns True provided the Both the objects Different memory Address.

=) Here "is not" operator returns False provided the Both the objects contains same Memory Address.

NOTE ① :- if two objects participates in DEEP copy then "is" operators returns True.

NOTE ② :- if two objects participates in SHALLOW copy then "is" operator returns False.

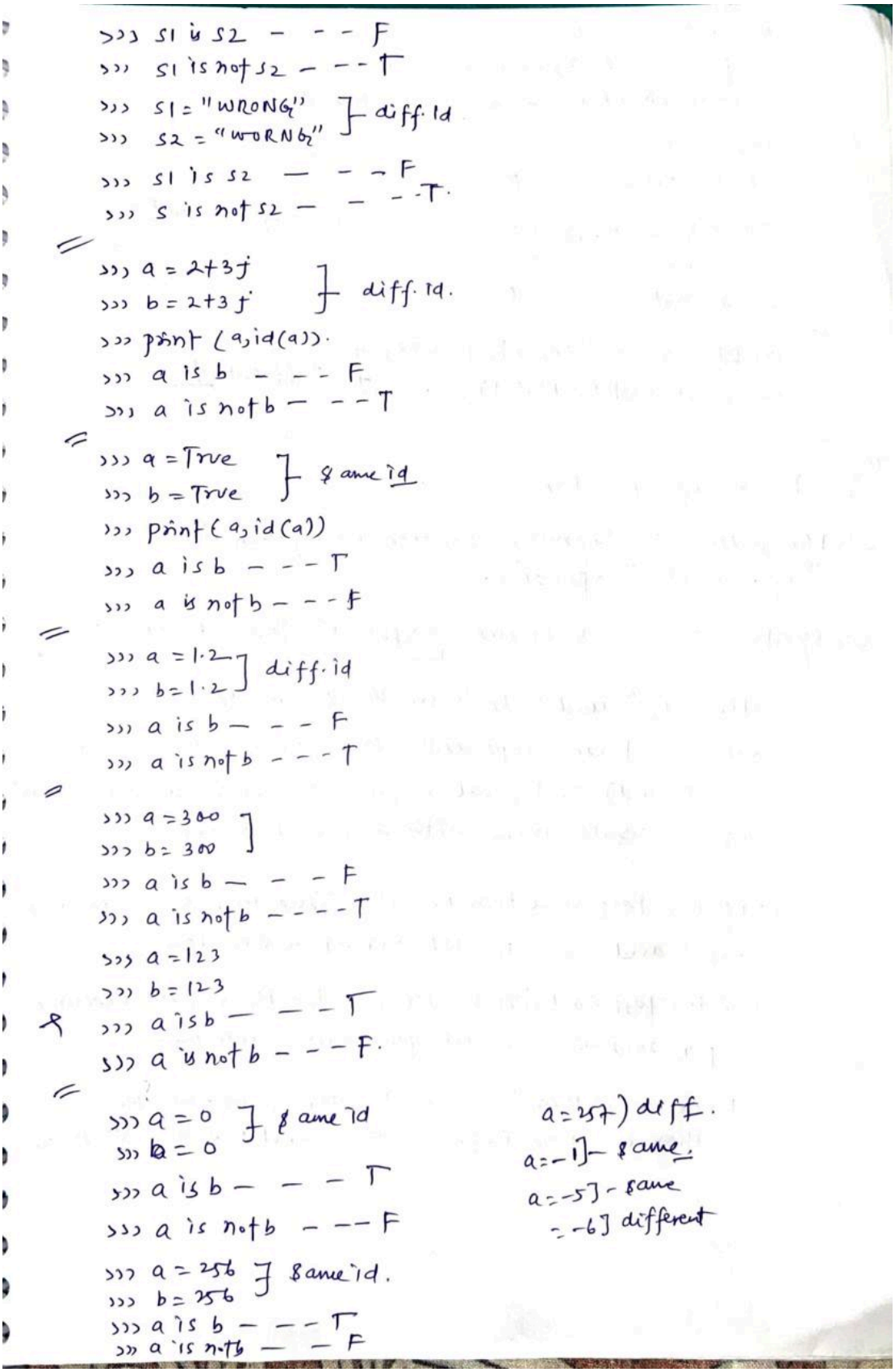
NOTE ③ :- if two objects participates in DEEP copy then "is not" operators returns False.

NOTE ④ :- if two objects participates in SHALLOW copy then "is not" operator returns True.

Examples :-

```
>>> a = None
>>> b = None
>>> print(a, id(a))  - - - None 140725686 52
>>> print(b, id(a))  - - None " " " 52.
>>> a is b  - - - T
>>> a is not b  - - F.
=
>>> d1 = {10: "Apple", 20: "Mango"}
>>> d2 = {10: "Apple", 20: "Mango"}.
>>> print(d1, id(d1))  ↑ same 408 - id.
>>> print(d2, id(d2))  264 - different id.
>>> d1 is d2  - - - F
>>> d1 is not d2  - - - T
```

```
>>> s1 = {10,20,30,40}
>>> s2 = {10,20,30,40}.
>>> print (s1, id(s1)) - - - - {40,10,20,30}   68.
>>> print (s2, id(s2)) - - - {40,10,20,30}     44  diff.id.
>>> s1 is s2    - - - F
>>> s1 is not s2 - - - T
=
>>> lst1 = [10, "Likit", 45.67]
>>> lst2 = [10, "Likit", 45.67]
>>> print (lst1, id(lst1)) - - - [10,'Likit',45.67]  24
>>> print (lst2, id(lst2)) - - - [10,'Likit',45.67]  96 diff.id.
>>> lst1 is lst2  - - - F
>>> lst1 is not lst2 - - - T
=
>>> lst1 = (10, "Likit", 45.67)  {diffent id}.
>>> lst1 is lst2  - - - F
>>> lst1 is not lst2 - - T.
=
>>> r1 = range (10,20)    {different id}.
>>> r1 is r2 - - - F
>>> r1 is not r2 - - - T.
=
>>> b1 = bytes (range (10,20))  ] different id.
>>> b2 = bytes (range (10,20))
>>> b1 is b2 - - - F
>>> b1 is not b2 - - - T
=
>>> s1 = "INDIA"        ] same id.
>>> s1 is s2 - - - T
>>> s1 is not s2 - - - F
>>> s1 = "THIS"         ] different add.
>>> s2 = "THSI"
>>> s1 is s2 - - - F
>>> s1 is not s2 - - - T
>>> s2 = "JUST"         ] different id.
>>> s1 = "JSUT"
>>> print (s1, id(s1)).
```



6.Membership operators:-

- in python programming,we have two types of membership operators:
- 1)in operator
- `syntax:- value in iterable_object`
- 2)not in operator

- syntax:- value not in iterable_object

In [1]:
```
s="python"
print(s,type(s))
```

```
python <class 'str'>
```

In [2]:
```
"p" in s
```

Out[2]: True

In [3]:
```
"o" in s
```

Out[3]: True

In [4]:
```
"n" not in s
```

Out[4]: False

In [5]:
```
"p" in s
```

Out[5]: True

In [6]:
```
"p" not in s
```

Out[6]: False

In [7]:
```
"t" in s
```

Out[7]: True

In [8]:
```
"t" not in s
```

Out[8]: False

In [15]:
```
s="PYTHON"
print(s,type(s))
"pyt" in s
```

```
PYTHON <class 'str'>
```

Out[15]: False

In [16]:
```
"pyt" not in s
```

Out[16]: True

In [17]:
```
"pyt" in s
```

Out[17]: False

In [18]:
```
"pto" in s
```

Out[18]: False

In [19]:
```
"pon" not in s
```

Out[19]: True

In [20]:
```
"pyt" not in s
```

Out[20]: True

In [21]:
```
"NOH" in s
```

Out[21]: False

In [22]:
```
"NOH" not in s[::-1]
```

Out[22]: False

In [23]:
```
s="PYTHON"
print(s,type(s))
```

```
PYTHON <class 'str'>
```

In [25]:
```
"HON"[::-1] in s[::-1]
```

Out[25]: True

In [26]:
```
"PTO" in s[::2]
```

Out[26]: True

In [27]:
```
"PTO" not in s[::2][::-1]
```

Out[27]: True

In [28]:
```
"PTO"[::-1] in s[::2]
```

Out[28]: False

In [29]:
```
lst=[10,"rossum",23.45,2+3j]
print(lst,type(lst))
```

```
[10, 'rossum', 23.45, (2+3j)] <class 'list'>
```

In [30]:
```
"ross" in lst
```

Out[30]: False

In [31]:
```
"ross" not in lst[1]
```

Out[31]: False

In [32]:
```
"ross" in lst[1]
```

Out[32]: True

In [33]:
```
"rossum"[::-1] not in lst[-3][::-1]
```

Out[33]: False

In [34]:
```
"rossum"[::-1] in lst[-3][::-1]
```

Out[34]: True

In [35]:
```
(2+3j) in lst[-1]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[35], line 1
----> 1 (2+3j) in lst[-1]

TypeError: argument of type 'complex' is not iterable
```

In [36]:
```
lst[-1] in lst
```

Out[36]: True

In [37]:
```
(2+3j) in str(lst[-1])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[37], line 1
----> 1 (2+3j) in str(lst[-1])

TypeError: 'in <string>' requires string as left operand, not complex
```

In [38]:
```
12 in "123"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[38], line 1
----> 1 12 in "123"

TypeError: 'in <string>' requires string as left operand, not int
```

In [39]:
```
"12" in "123"
```

Out[39]: True

In [40]:
```
d={10:"apple",20:"mango",30:"kiwi"}
"apple" not in d
```

Out[40]: True

In [41]:
```
for val in d:
    print(val)
```

```
10
20
30
```

In [42]:
```
10 in d
```

Out[42]: True

In [45]:
```
20 not in d
```

Out[45]: False

In [47]:
```
20 in d.keys()
```

Out[47]: True

In [48]:
```
d[10][::-1] in d.get(10)
```

Out[48]: False

In [49]:
```
"1" not in "123"[1]
```

Out[49]: True

In [50]:
```
"mis" in "mississippi"
```

Out[50]: True

In [52]:
```
"iip" not in "mississippi"[::-1]
```

Out[52]: True

In [54]:
```
"ippi"[-1] not in "mississippi"
```

Out[54]: False

7.identity operators:

- in python programming we have two types of identity operators
- 1)is
- syntax:- object1 is object2
- 2)is not
- syntax:- object1 is not object2

In [56]:
```
a=None
b=None
print(a,id(a))
print(b,id(b))
```

```
None 140711911426176
None 140711911426176
```

In [57]:
```
a is b
```

Out[57]: True

In [58]:
```
a is not b
```

Out[58]: False

In [59]:
```
d1={10:"apple",20:"mango"}
d2={10:"apple",20:"mango"}
print(d1,id(d1))
print(d2,id(d2))
```

```
{10: 'apple', 20: 'mango'} 2967969279360
{10: 'apple', 20: 'mango'} 2967968936192
```

In [60]:
```
d1 is d2
```

Out[60]: False

In [61]:
```
d1 is not d2
```

Out[61]: True

In [62]:
```
s1={10,20,30,40}
s2={10,20,30,40}
print(s1,id(s1))
print(s2,id(s2))
```

```
{40, 10, 20, 30} 2967968943008
{40, 10, 20, 30} 2967968945024
```

In [63]:
```
s1 is s2
```

Out[63]: False

In [64]:
```
s1 is not s2
```

Out[64]: True

In [65]:
```
lst1=[10,"likit",45.67]
lst2=[10,"likit",45.67]
print(lst1,id(lst1))
print(lst2,id(lst2))
```

```
[10, 'likit', 45.67] 2967955105856
[10, 'likit', 45.67] 2967969546496
```

In [66]:
```
lst1 is lst2
```

Out[66]: False

In [67]:
```
lst1 is not lst2
```

Out[67]: True

In [68]:
```
ls1=(10,"likit",45.67)
ls2=(10,"likit",45.67)
print(ls1,type(ls1),id(ls1))
print(ls2,type(ls2),id(ls2))
```

```
(10, 'likit', 45.67) <class 'tuple'> 2967969374400
(10, 'likit', 45.67) <class 'tuple'> 2967969555456
```

In [69]:
```
ls1 is ls2
```

Out[69]: False

In [70]:
```
ls1 is not ls2
```

Out[70]: True

In [73]:
```
r1=range(10,20)
r2=range(10,20)
print(r1,type(r1),id(r1))
print(r2,type(r2),id(r2))
```

```
range(10, 20) <class 'range'> 2967954698000
range(10, 20) <class 'range'> 2967951479680
```

In [74]:
```
r1 is r2
```

Out[74]: False

In [75]:
```
r1 is not r2
```

Out[75]: True

In [76]:
```
b1=bytes(range(10,20))
b2=bytes(range(10,20))
print(b1,type(b1),id(b1))
print(b2,type(b2),id(b2))
```

```
b'\n\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13' <class 'bytes'> 2967957409488
b'\n\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13' <class 'bytes'> 2967957407904
```

In [77]:
```
b1 is b2
```

Out[77]: False

In [78]:
```
b1 is not b2
```

Out[78]: True

In [79]:
```
s1="india"
s1 is s2
```

Out[79]: False

In [80]:
```
s1 is not s2
```

Out[80]: True

In [81]:
```
s1="this"
s2="thsi"
print(s1,id(s1))
print(s2,id(s2))
```

```
this 2967858520224
thsi 2967957452432
```

In [82]:
```
s1 is s2
```

Out[82]: False

In [83]:
```
s1 is not s2
```

Out[83]: True

In [84]:
```
a=True
b=True
print(a,b)
```

```
True True
```

In [85]:
```
a is b
```

Out[85]: True

In [86]:
```
a is not b
```

Out[86]: False

In [87]:
```
a,b=300,300
print(a,id(a))
print(b,id(b))
```

```
300 2967968197552
300 2967968197552
```

In [88]:
```
a is b
```

Out[88]: True

In [89]:
```
a is not b
```

Out[89]: False

In [90]:
```
a,b=23456,23456
print(a,b,id(a),id(b))
```

```
23456 23456 2967968198960 2967968198960
```

In [91]:
```
a is b
```

Out[91]: True

In [92]:
```
a is not b
```

Out[92]: False

In [93]:
```
lst1,lst2=[10,20],[10,20]
print(lst1,lst2,id(lst1),id(lst2))
```

```
[10, 20] [10, 20] 2967969381696 2967969354752
```

In [95]:
```
lst1 is lst2
```

Out[95]: False

In [97]:
```
lst is not lst2
```

Out[97]: True

In [ ]:
```

```